

Lab Exercise 10

Lab Question and Answer: File Input and Output Using Sentinel Loops in C

LAB QUESTION

Write a C program that uses text file handling and user-defined functions to manage student records. The program must apply sentinel-controlled loops when writing data to a file and when reading data from a file.

Your program must include **two user-defined functions** as described below.

Function 1: writeStudentData()

This function should:

1. Create or open a text file named **student.txt** in **write** mode.
2. Repeatedly prompt the user to enter student records using a **sentinel-controlled loop**. Use a sentinel value of -1 to stop data entry.
Input the following data:
 - Student ID (integer)
 - Student Name (string)

The sentinel condition for terminating data entry is:

 - Student ID equals **-1**
(When the user enters -1, no data should be written and the loop must stop.)
3. Write each valid student record to the file in the following format (one record per line):

```
<student_id> <student_name>
```
6. Display an appropriate error message if the file cannot be opened.
7. Close the file after all data has been written.

Function 2: displayStudentData()

This function should:

1. Open the **student.txt** file in read mode.
2. Read and display all student records using a **sentinel-controlled loop**.
3. The sentinel condition must be based on the **end-of-file (EOF)** or the return value of `fscanf()`.
4. Display each record in a clear and readable format.

5. Display an error message if the file cannot be opened.
6. Close the file after reading is completed.

Main Function Requirements

The `main()` function should:

1. Call the `writeStudentData()` function to store student records.
2. Call the `readStudentData()` function to retrieve and display all stored records.
3. Terminate the program correctly.

Sample Input/Output

```
D:\Ar Bukhary\CCCH23-Prob5 >
Enter Student ID (-1 to stop): 12345
Enter Student Name: BAHARUDIN
Enter Student ID (-1 to stop): 23214
Enter Student Name: OSMAN
Enter Student ID (-1 to stop): 24432
Enter Student Name: HAMIZA
Enter Student ID (-1 to stop): 32321
Enter Student Name: HAKIMI
Enter Student ID (-1 to stop): 45323
Enter Student Name: BADRUL
Enter Student ID (-1 to stop): 35262
Enter Student Name: SALMA
Enter Student ID (-1 to stop): 26632
Enter Student Name: FATIMA
Enter Student ID (-1 to stop): -1

Reading data from file:
Student ID : 12345
Student Name : BAHARUDIN

Student ID : 23214
Student Name : OSMAN

Student ID : 24432
Student Name : HAMIZA

Student ID : 32321
Student Name : HAKIMI

Student ID : 45323
Student Name : BADRUL
```

```
Enter Student ID (-1 to stop): 2222
Enter Student Name: OSMAN
Enter Student ID (-1 to stop): 3333
Enter Student Name: FAROUQ
Enter Student ID (-1 to stop): 4444
Enter Student Name: TURAI
Enter Student ID (-1 to stop): 5555
Enter Student Name: SALMA
Enter Student ID (-1 to stop): -1
```

```
Reading data from file:
Student ID : 1111
Student Name : BAHARUDIN
```

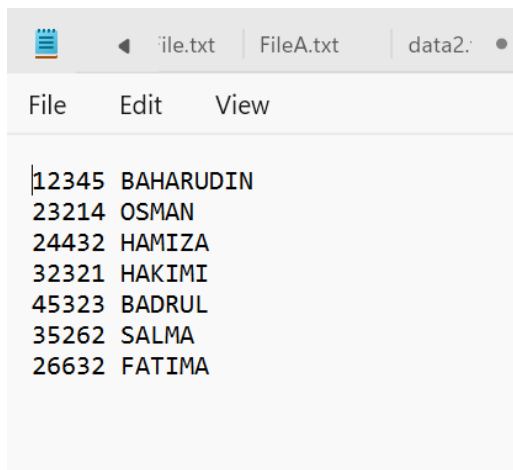
```
Student ID : 2222
Student Name : OSMAN
```

```
Student ID : 3333
Student Name : FAROUQ
```

```
Student ID : 4444
Student Name : TURAI
```

```
Student ID : 5555
Student Name : SALMA
```

Upon successful execution of the program, the file *student.txt* will be created, and its contents will be as shown below:



The screenshot shows a text editor window with a menu bar (File, Edit, View) and a toolbar. The file name 'student.txt' is visible in the title bar. The contents of the file are as follows:

```
12345 BAHARUDIN
23214 OSMAN
24432 HAMIZA
32321 HAKIMI
45323 BADRUL
35262 SALMA
26632 FATIMA
```